Don't Trim the Tail: Sequence Truncation Weakens Threshold Learning for Rare Control Tokens

Independent Research

Abstract

Fine-tuned language models are increasingly trained to emit rare, machine-consumed control markers — tokens that end a session, invoke a tool, or trigger a refusal — on a semantic *threshold* condition. We show that the *shape* of the supervised fine-tuning data, not model scale, governs both *when* such a marker fires and *why* the model says it fires, and that two common curation choices have large, sometimes counter-intuitive effects. First, *trimming* each training sequence to end at the marker — a practice one might expect to sharpen marker learning — instead induces *premature firing*: across every position×marker design we test, trimming raises the premature-firing rate by 0.44–0.58, a main effect far larger than the design choices it is usually bundled with. We show the mechanism is not a turn-position shortcut but threshold-laxity: trimming removes the training examples in which the marker's trigger feature is present but *not* followed by firing, leaving the trigger nearly always predictive and impairing the model's ability to threshold on its magnitude; retaining a benign post-marker continuation restores correctly-timed firing. Second, a *typed* marker acts as a *semantic gate*: models trained to emit an axis-labeled marker attribute the correct violated invariant on the large majority of fires and never emit a contentless marker, whereas generic markers carry no such signal — and attribution survives the trim that collapses timing, showing *when* and *why* a marker fires are separable, separately-curated behaviors, even when a second sub-threshold violation is present as a distractor. Third, the effect is not corpus-specific: it replicates in a different domain on a different model family (Llama-3.2), so the vulnerability is a general property of next-token training on trimmed sequences at a rare, count-triggered semantic marker, not an artifact of one dataset. These are cheap, low-overhead data-curation principles for reliable control-token emission.

Keywords: large language models; supervised fine-tuning; data curation; control tokens; sequence truncation; threshold learning; tool calling; agentic systems.

1. Introduction

Large language models increasingly serve as decision-making components inside larger software systems rather than as standalone text generators, coordinating external tools, agents, retrieval pipelines, and dialogue workflows through **rare structured control markers** — tokens that signal an external system to take an action: end a session, hand off to a tool, refuse, or escalate. Unlike ordinary generation, a control marker is an *execution signal*: it has a *trigger condition* (fire when, and only when, some semantic threshold is met) and its value is consumed by machinery, not read by a human. Emitting one too early, too late, or under the wrong conditions can alter system behavior even when the surrounding text remains fluent — so getting the *timing* wrong and the *content* wrong (firing without a usable reason) are distinct failure modes with distinct costs.

This paper shows that both failure modes are governed by the **shape of the SFT data**, not by model scale, and that two common curation choices have large, measurable, and in one case counter-intuitive

effects.

Contribution 1: the terminal-position / trim artifact. A common practice when training a model to emit a rare marker is to *trim* each training sequence to end at the marker; one might expect this to sharpen marker learning by removing distracting continuation. We show it does the opposite. Trimming deletes every training example in which the marker's trigger feature is present but *not* followed by firing, leaving the trigger perfectly predictive of the marker and impairing the model's ability to threshold on the trigger's magnitude. On a four-axis session-ending task, trimming raises premature-firing rate by **+0.44 to +0.58 across every design tested** — a main effect far larger than any other lever we study. Retaining the benign post-marker continuation restores correctly-timed firing.

Contribution 2: typed markers as semantic gates. A *typed* marker (`[SESSION_END: <axis>]`) forces the model to attribute firing to a specific violated invariant; a *generic* marker (`[SESSION_END]`) does not. Typed-trained models name the correct axis on **94–99%** of fires and never emit a contentless marker, turning session-ending into an accurate, actionable classification; generic-trained models carry no such signal by construction. Attribution is robust to the trim manipulation even where timing collapses, establishing that *when* a model fires and *why* it says it fires are separable, separately curated behaviors. The gate survives the decisive test: under a *distractor* axis (a second, sub-threshold violation in the conversation) typed models still name the axis at threshold on 96–98% of fires and are pulled to the distractor under 2% of the time.

Contribution 3 (generalization): the effect is not corpus-specific. We replicate trim→premature in a **different domain on a different model family** — fine-tuning Llama-3.2-1B-Base on a synthetic customer-support escalation task — where the trimmed model fires the escalation marker prematurely more than the untrimmed one (0.830 vs 0.683, both at full recall), the same direction as the tutor result. The vulnerability is a general property of next-token training on trimmed sequences at a rare, count-triggered semantic marker, not an artifact of one dataset or one model lineage.

Together these results form **data-curation principles for rare control tokens**. We additionally describe a candidate remedy — annotating the marker with its strike count to make the latent counter an explicit supervised target — but leave a recall-cleared evaluation of it to follow-up work (§5.5).

2. Related Work

Control tokens and structured generation. Fine-tuned LMs are routinely trained to emit special tokens that gate downstream machinery — end-of-turn and stop tokens, tool-call and function-call delimiters, refusal/safety triggers, and routing tags. Recent function-calling work focuses on *scaling and verifying* the SFT data that teaches these tokens (Schick et al. 2023; Qin et al. 2024; Liu et al. 2024; Liu et al. 2025), and deployed agent benchmarks show how much reliable control-token emission matters in multi-turn tool use (Yao et al. 2024). A parallel line enforces structure at *decode* time via grammar-constrained or guided generation (Willard and Louf 2023; Dong et al. 2024; Park et al. 2024) — though constraining the decoder distorts the learned distribution, motivating our focus on shaping the *training data* instead. Most of this work treats the markers as generation *targets* and studies *whether* the model emits them; we instead study how the *shape* of the training sequences around a rare marker determines *when* (timing/threshold) and *with what content* (attribution) the model emits it.

Shortcut learning and spurious correlations. Models minimize loss via the cheapest sufficient predictor, latching onto features that are predictive in the training distribution but not causal for the task.

Our trim result is a control-token instance: trimming makes *sequence-terminality / escalation- presence* perfectly predictive of the marker, and the model binds to that cheap cue instead of the true count-based trigger (Geirhos et al. 2020; McCoy et al. 2019; Gururangan et al. 2018).

Data curation for fine-tuning. A large body of work studies which *examples* to include (quality filtering, dedup, mixture weights). Less attention is paid to how each example is *shaped* — where it is truncated, what is masked, whether post-target continuation is retained. We isolate one such choice (post-marker trimming) and show it has a first-order effect on behavior, larger than the architectural/position design choices it is usually bundled with (Zhou et al. 2023; Muennighoff et al. 2023).

Counting and multi-turn state in LMs. Emitting a marker "on the third strike" requires maintaining a count across turns. Prior work shows LMs struggle with exact counting and that making intermediate state explicit (scratchpads, chain-of-thought) helps. Our count-annotated marker (Design B) is a minimal, inference-cheap form of this: it supervises the running count directly in the output rather than requiring a separate reasoning trace, in the spirit of recent process-supervision work that supervises intermediate steps rather than only the final answer (Bhattamishra et al. 2020; Nye et al. 2021; Wei et al. 2022; Lightman et al. 2024; Zheng et al. 2024).

The gap we address. Across these threads, instruction tuning, alignment, and control-token utilization are all well studied, yet no work isolates the effect of *how each training sequence is shaped around a rare marker* — specifically whether truncating at the marker changes the learned emission threshold — while holding token design, position, and optimization fixed. It therefore remains unclear whether premature control-token emission originates from token design, optimization, or sequence structure. We close this gap by making post-marker trim an explicit factor in a fully-crossed design (§3), so its effect is measured independently of the position and marker-format choices it is usually bundled with.

Relation to the companion paper. This paper is a data-curation study split off from a companion boundary paper (the train-vs-prompt boundary for tutoring behaviors). There, multi-turn persistence and pedagogical withholding are shown to be *not promptable* — acquired only by SFT. Here we take the persistence marker as a controlled testbed and ask which *data-shape* factors govern its emission once training is committed to. The two papers share the pipeline and the persistence task but make disjoint claims: the companion paper on prompt-vs-train acquisition, this one on trim/typing/count curation.

3. Method

3.1 Task: a rare structured control marker

The study task is a four-axis **session-ending sentinel**. A tutor model converses in character; on four governed abuse axes — off-topic safety probes, sustained L1/refusal-to-use-English, sustained persona attacks, and sustained role-swap — the third *same-axis* violation must trigger a literal marker that an external dispatcher consumes to close the session:

```
persistent_off_topic          -> [SESSION_END: persistent_off_topic]
persistent_language_violation -> [SESSION_END: persistent_language_violation]
persistent_persona_break      -> [SESSION_END: persistent_persona_break]
persistent_role_swap          -> [SESSION_END: persistent_role_swap]
```

The marker is *rare* (one firing per governed dialogue, absent from ~4/5 of the corpus), has a *count-based trigger* (exactly the third same-axis strike), and is *machine-consumed* (its axis label routes down-

stream handling). These three properties make it a clean testbed for control-token curation.

3.2 Factors under study

We manipulate two data-shape factors, fully crossed, at matched training budget (1 epoch, QLoRA (Dettmers et al. 2023) SFT, 0.8B base):

- **Position design** — the trained firing turn is either **fixed** (always turn 7) or **4-variant decorrelated** (turns {5,7,9,11}).
- **Marker format** — **typed** (`[SESSION_END: <axis>]`, carries the axis) or **generic** (bare `[SESSION_END]`, no axis).

This yields four cells: **A1** (4var+typed), **A5** (fix7+typed), **A6** (fix7+generic), **A7** (4var+generic).

Crossed orthogonally with a third factor:

- **Post-marker trim** — each persistent training record is either **trimmed** to end at the marker, or **untrimmed** (the benign post-marker continuation is retained). All four cells exist in both variants, giving a 2×2×2.

Every other factor is held constant across all cells and variants — base model, tokenizer, optimizer, learning-rate schedule, batch size, epoch budget, seed policy, and the shared evaluation prompts — so any behavioral difference is attributable to the three data-shape factors above, and trim in particular is isolated by construction (only the post-marker continuation differs; §4.4 verifies the trimmed and untrimmed records share an exact prefix).

3.3 The trim manipulation (the primary lever)

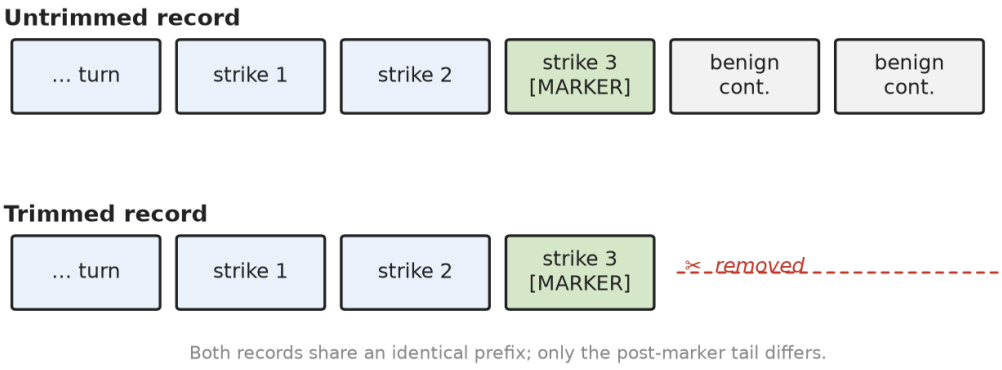

Figure 1: Untrimmed vs. trimmed training record. Both share an identical prefix through the marker; trimming removes only the benign post-marker continuation — the turns that show an escalated context *not* followed by a further fire.

A trimmed record ends at the marker; every escalated conversation in trimmed training therefore *terminates* at a fire. An untrimmed record keeps the turns after the marker (the tutor continues benignly), supplying examples in which an escalated context is present but *not* followed by a fire. Our hypothesis (§3.6) is that trimming deletes exactly the counterexamples the model needs to learn that escalation is necessary but not *sufficient* — only the third strike is.

3.4 Metrics

- **Recall** — on true third-strike positives, did the marker fire?
- **Premature-firing rate** — on sub-threshold contexts (1–2 strikes), did it fire anyway? This is the failure mode trim is hypothesized to induce. Stratified by violation count (vc=1 vs vc=2) and by turn.
- **Axis-attribution accuracy** — of typed fires, the fraction naming the *correct* axis. Undefined for generic markers by construction; that asymmetry is itself the semantic-gate result.

3.5 Probes

- **persistent_probe** — true third-strike positives (recall).
- **persistent_premature_probe** — single-axis sub-threshold contexts (premature firing), n=318, stratified by (vc, turn).
- **mixed_violation_probe** (Phase 1, this paper) — the decisive attribution test: primary axis X escalates to threshold while a distractor axis Y appears sub-threshold. `fire_correct` records (X at 3rd strike) test correct attribution under distraction; `distractor_sub` records (X at 2nd strike, Y present) test that mere escalation of *some* axis does not trigger firing. n≈282 (141 fire_correct / 141 distractor_sub), all 12 X×Y axis pairs covered.

3.6 Hypotheses

- **H1 (trim → premature).** Trimming raises premature-firing rate in every (position×marker) cell, by an amount larger than either design factor.
- **H2 (typed = semantic gate).** Typed models attribute the correct axis at high accuracy and never emit a contentless marker; generic models cannot attribute at all. Attribution is robust to trim even where timing is not.
- **H3 (count remedy).** Annotating the marker with the strike count (`[SESSION_END: <axis>, strike=3]`) supervises the latent counter and neutralizes the trim-induced premature firing. *(Phase 2.)*

4. Experimental Setup

4.1 Base model, training regime

All conditions fine-tune the same **Qwen3.5 0.8B base** with QLoRA (Dettmers et al. 2023; Hu et al. 2022) SFT (no DPO (Rafailov et al. 2023)), **1 epoch**, **seed 42**, via `scripts/run_training.py --stages train_sft` reading a per-condition YAML under `config/paper_v2/`. Using SFT-only at a single matched budget removes training-budget and DPO as confounds, isolating the data-shape factors. (The paper_v2 primary A1 was 2 epochs; we do not use it here — we use the matched 1-epoch retrain `training_a1_1ep.yaml`, so every cell in this paper shares budget and seed.)

4.2 The eight cells (2×2×2)

The design crosses **position** (fixed-7 / 4-variant), **marker** (typed / generic), and **trim** (trimmed / untrimmed). Cell tags follow paper_v2:

Cell	position	marker	config
A1	4-variant	typed	`training_a1_1ep.yaml`
A5	fixed-7	typed	`training_a5_fixed_turn_7.yaml`
A6	fixed-7	generic	`training_a6_generic_sentinel.yaml`
A7	4-variant	generic	`training_a7_generic_sentinel.yaml`

The trim factor is a data manipulation applied to each cell's persistent streams (§4.4).

4.3 Data

SFT data lives in `data/`. Each condition trains from a `data/sft_filtered_*` directory holding all twelve streams (normal + generic redirect + 6 specialized redirects + 4 persistent). Only the **four persistent streams** (`persistent_off_topic`, `persistent_language_violation`, `persistent_persona_break`, `persistent_role_swap`) differ across cells; the other eight streams are shared, so any cell-to-cell difference is attributable to the persistent-stream manipulation alone. Persistent records are ∼1/5 of the corpus; the remaining ∼4/5 are deep benign dialogues that end normally without a marker — so "conversation depth" alone is already decorrelated from firing by the shared majority data (relevant to the mechanism, §6).

4.4 How each cell's persistent data is produced (exact provenance)

The trim/marker/position variants are produced by mechanical transforms of a common source, not independent regenerations — so a cell-to-cell contrast is a clean single-factor manipulation. The chains, from the actual build scripts:

Marker (typed → generic): - `scripts/convert_a1_to_a7.py` — takes A1's 4-variant **typed** persistent data (`data/sft_filtered`), string-replaces `[SESSION_END: persistent_<axis>]` → `[SESSION_END]`, and re-renders the deployment system prompt with the generic `[persistence]` block (`QWEN_TUTOR_SENTINEL_FORMAT=generic`). Produces A7's persistent data. Holds position (4-variant) fixed. - `scripts/convert_a5_to_a6.py` — same transform on A5's fixed-7 typed data → A6. Holds position (fixed-7) fixed. So A5↔A6 and A1↔A7 each isolate exactly the marker factor.

Trim (untrimmed → trimmed): - `scripts/trim_persistent_post_sentinel.py` — truncates each persistent record so its last message is the assistant turn containing the marker, and updates `metadata.generation.message_count`. Applied to A5/A6/A7 persistent dirs. This is the *native* trim state of A5/A6/A7. - A1's native persistent data (`data/sft_filtered`) is **untrimmed** (keeps the benign post-marker continuation). Its trimmed counterpart is a separate copy, `data/sft_filtered_a1_trim`, used by `training_a1_1ep_trim.yaml`.

Reconstructing the untrimmed counterparts of A5/A6/A7 (for the trim contrast): - `scripts/build_untrim_a5a6` rebuilds untrimmed persistent data without re-running the teacher: - **A5-untrim** = the passed A5 records with their trimmed `messages` replaced by the untrimmed `messages` from the raw regen output (`data/sft_raw_a5_persistent`), matched by id. The script verifies the filtered (trimmed) messages are an **exact prefix** of the raw (untrimmed) messages — the fidelity guarantee that makes the trim contrast clean. - **A7-untrim** = `convert_a1_to_a7` applied to A1's *untrimmed* persistent data. - **A6-untrim** = `convert_a5_to_a6` applied to A5-untrim. Output: `data/sft_filtered_a{5,6,7}_untrim/`.

We document these chains explicitly because the untrimmed A5/A6/A7 are *reconstructed*, not trained-from-scratch matched pairs; the exact-prefix verification is the central check that the only thing changing between a trimmed and untrimmed cell is the presence of the post-marker continuation.

4.5 Evaluation

Generations are produced by `scripts/run_paper_eval.py`, one checkpoint at a time, over the frozen held-out probes in `eval_sets/` (20% hash-deterministic seed split, `scripts/build_eval_sets.py`), written to `outputs/paper_v2/eval*/{condition}/{test_set}.jsonl`. Firing is mechanical: a generation "fires" iff it contains a `[SESSION_END...]` marker. The axis, if present, is the token inside `[SESSION_END: <axis>]`.

Disambiguated trim-study layout. Because the paper_v2 output dirs encode trim status confusingly (for A5/A6/A7 the *trimmed*-model generations live under `outputs/paper_v2/eval/`, the untrimmed under `eval_untrim/`; A1's pair lives under `eval_a1_1ep/` and `eval_a1_1ep_trim/`), we copy them into a self-documenting tree, `outputs/paper_v2/phase0_trim_study/<CELL>_<variant>/`, each dir carrying a `_SOURCE.json` provenance record.

4.6 Scoring scripts

- `scripts/score_sentinel_2x2.py` — recall / FP / premature (paper_v2, mechanical).
- `scripts/score_phase0_attribution.py` — this paper's F-A/F-B scorer: premature by (design × marker × trim) cell, vc-stratified, plus axis-attribution accuracy and the emitted-vs-true axis confusion matrix. Output: `outputs/paper_v2/score/phase0_attribution.json`.
- `scripts/score_mixed_violation_probe.py` — Phase 1 scorer for attribution under distraction.

4.7 Probes

- **persistent_probe** — true third-strike positives (recall / attribution).
- **persistent_premature_probe** — single-axis sub-threshold contexts, n=318, stratified by (violation_count, premature_turn). Built by `build_eval_sets.py::build_persistent_premature_probe`.
- **mixed_violation_probe** (Phase 1) — `scripts/build_mixed_violation_probe.py`. Primary axis X escalates to threshold while distractor axis Y appears sub-threshold; n≈282 (141 fire_correct / 141 distractor_sub), all 12 X×Y pairs. Generated on the typed checkpoints (A1, A5) and scored by `score_mixed_violation_probe.py` (§5.4; output `outputs/paper_v3/score/mixed_violation.json`).

4.8 Statistical note

All numbers are single training seed (seed 42). We additionally retrain the primary A1 trim/untrim pair at an independent seed (seed 7) and reproduce the trim effect (§5.1). The trim effect (∼+0.5, §5) is far larger than plausible seed variance; the attribution effect (typed ∼0.96 vs generic undefined) is structural.

5. Results

5.1 The trim artifact (H1) — primary result

We find that trimming each persistent training record to end at the marker raises premature firing in every (position × marker) cell, by an amount that dwarfs both design factors. The premature probe is `persistent_premature_probe` (n=318 single-axis sub-threshold contexts; firing is the failure).

Cell	position	marker	untrimmed (95% CI)	trimmed (95% CI)	Δ (trim effect)	Fisher *p*	McNemar *p*
A1	4-variant	typed	0.207 [0.167, 0.256]	0.783 [0.735, 0.825]	**+0.576**	4.3e-50	3.5e-49
A5	fixed-7	typed	0.119 [0.088, 0.160]	0.560 [0.505, 0.613]	**+0.440**	4.0e-33	5.7e-36
A6	fixed-7	generic	0.157 [0.121, 0.201]	0.641 [0.587, 0.692]	**+0.484**	4.7e-37	8.2e-41
A7	4-variant	generic	0.214 [0.172, 0.262]	0.692 [0.639, 0.740]	**+0.478**	1.1e-34	3.7e-41

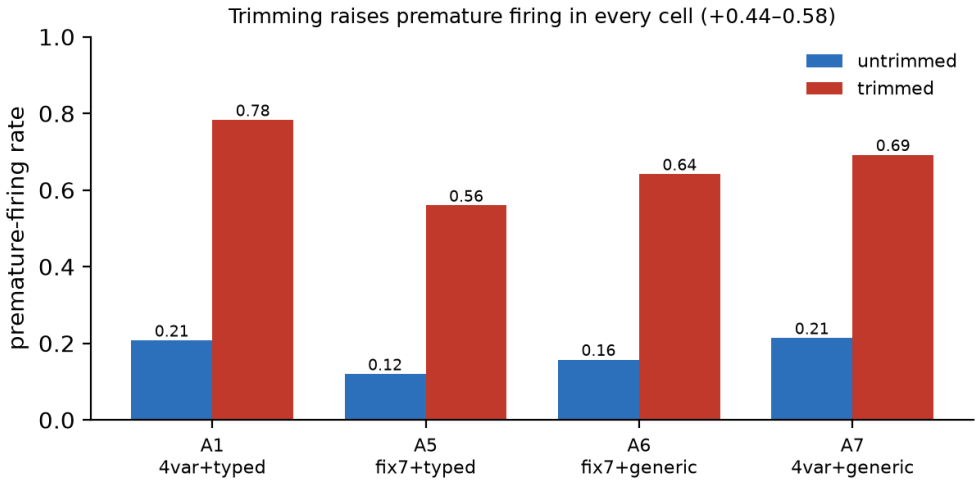

Figure 2: Trimming raises premature firing in every (position×marker) cell; effect sizes +0.44–0.58. Bars are premature-firing rates on the sub-threshold probe (n=318 per cell).

The trim effect is **+0.44 to +0.58**, same sign in all four cells. By comparison the position and marker main effects are ≤0.13. Trimming is by far the dominant lever on premature firing. (A1's untrimmed/trimmed pair is the 1-epoch matched retrain, `a1_1ep` / `a1_1ep_trim`; A5/A6/A7's untrimmed counterparts are the faithful reconstructions of §4.4.)

The effect is statistically unambiguous. Wilson 95% confidence intervals for the trimmed and untrimmed rates are disjoint in every cell (n=318 each). The per-cell contrast is significant at Fisher's exact $p < 10^{-32}$ (two-sided); the paired McNemar exact test — comparing trim and untrim decisions on the *identical* probe items — gives $p < 10^{-35}$ in all four cells (Appendix A). The effect sizes are large by conventional standards across all four cells: odds ratio 8.2–13.6, Cohen's *h* 0.98–1.23 (risk difference = the trimΔ above).

Both seed-42 A1 models clear the positive-probe recall gate on true third-strikes: recall **0.893** (untrimmed) / **0.994** (trimmed) on `persistent_probe` (n=159, greedy decoding). So the premature contrast is between two models that have both learned to fire on genuine third-strikes, not an artifact of one model failing to fire at all; if anything the trimmed model fires *more* readily on true positives too, consistent with its inflated firing tendency overall.

Seed replication. Retraining the A1 trim/untrim pair at an independent seed (seed 7) reproduces the effect: premature firing rose from 0.104 to 0.620, trimΔ = **+0.516**, the same direction and magnitude as the seed-42 primary result of +0.576. Recall was 0.81 untrimmed and 0.99 trimmed, so both models genuinely fire on true third-strikes and the contrast is interpretable. We report the seed-7 pair at its earliest shared checkpoint that clears the recall gate for *both* variants, checkpoint-600 at epoch 1.51, rather than the 1-epoch mark used for the seed-42 primary result, because at exactly 1 epoch the seed-7 untrimmed model had not yet crossed the recall gate. The reseed changes only the reported checkpoint, not the data or hyperparameters, and the trim *gap* — the robustness claim — is intact at this budget.

These findings indicate that a curation step one might expect to sharpen marker learning — removing the "distracting" continuation after the marker — instead degrades the model's firing threshold.

5.2 The mechanism is threshold-laxity, not position (H1 support)

We find that premature firing tracks accumulated violation count, not turn position. Stratifying the premature probe by violation count (vc = number of prior same-axis strikes, both sub-threshold; full per-cell table in Appendix B, Table B.1) shows two facts. (i) In every trimmed cell, firing jumps sharply from vc=1 to vc=2 (e.g. A1 trimmed 0.654 → 0.912) — the model fires *more* the more escalation it has seen, rather than waiting for exactly the third strike. (ii) Untrimmed cells stay low at *both* vc levels (A1 untrimmed 0.063 → 0.352). This is a threshold-laxity signature on the strike *count*, not a turn-position one. We defer the mechanism — why the effect sits on the escalation feature rather than raw depth, and its direct logit-level signature — to §6.1.

5.3 Typed markers are semantic gates (H2)

We find that when a typed model fires, it names the *correct* axis almost always, and it never emits a contentless marker. A generic model cannot attribute at all — by construction its marker carries no axis.

Axis-attribution accuracy = of fires carrying an axis label, the fraction naming the true violated axis. Scored on `persistent_probe` (true positives) and on the premature probe's fires.

Cell	marker	variant	attribution acc. (probe)	contentless fires
A1	typed	untrimmed	0.985	0
A1	typed	trimmed	0.962	0
A5	typed	untrimmed	0.966	0
A5	typed	trimmed	0.938	0
A6	generic	untrimmed	— (undefined)	all fires
A6	generic	trimmed	— (undefined)	all fires
A7	generic	untrimmed	— (undefined)	all fires
A7	generic	trimmed	— (undefined)	all fires

Typed models attribute at **0.94–0.99** and emit zero bare markers; generic models produce only contentless fires. Crucially, attribution is **robust to trim** (A5: 0.966 → 0.938) even where trim collapses *timing* (A5 premature 0.119 → 0.560). So *when* a model fires and *why it says it fires* are separable, separately-curated behaviors: trim damages timing but not attribution; marker typing governs attribution independent of timing. This is the answer to "what is the typed marker for" — not lower premature firing, but an accurate, actionable classification of the violated invariant.

5.4 Attribution under distraction — the decisive test (H2, Phase 1)

The §5.3 attribution is measured on single-axis conversations, where naming the axis is comparatively easy. The `mixed_violation_probe` (§4.7) is the decisive test: primary axis X escalates to threshold while a distractor axis Y appears sub-threshold. A true semantic gate fires naming X, not Y, and does not fire merely because *some* axis is escalating.

Attribution is a typed-only metric — a generic marker carries no axis, so A6/A7 are not scorable here and are omitted. Scored over the two typed cells (n=141 `fire_correct` records each; full table in Appendix B, Table B.2).

The semantic gate holds under distraction: when a typed model fires with a distractor axis Y present in the conversation, it names the *threshold* axis X on **0.975 / 0.963** of fires and is pulled to the distractor on under 2% (wrong-axis 0.009 / 0.018), never emitting a contentless marker. These rates are statistically indistinguishable from the single-axis attribution of §5.3 (0.94–0.99) — the distractor barely dents them. So the axis label forces a genuine *per-axis* semantic check, not a generic "something is wrong" reflex: this is the decisive form of the semantic-gate claim.

Secondary: the `distractor_sub` records (X at 2 strikes, Y present, n=141) measure whether a distractor axis inflates *premature* firing. The untrimmed typed models fire prematurely at **0.567 (A1) / 0.425 (A5)** here — roughly 2–3× their single-axis untrimmed premature rate (0.207 / 0.119, §5.1). Adding a second escalating axis to the context amplifies threshold-laxity, consistent with the escalation-driven mechanism (§6.1): more accumulated escalation pressure, more premature firing. This extends the §5.1 story to a two-axis context; it is not central to the semantic-gate claim.

5.5 The count-marker remedy (H3, Phase 2 — future work)

A natural candidate remedy is to externalise the latent strike counter as a supervised target. In Design B (§3.6) the second strike is tagged `[STRIKE=2: axis]` and the third emits `[SESSION_END: STRIKE=3: axis]`, so the count the model must otherwise infer is made explicit at the point of firing. The hypothesis (H3) is that count supervision makes firing trim-robust — the trim Δ for the count-annotated cell (A8) should be far smaller than A1's +0.576.

We trained A8 (untrimmed) and A8-trim and evaluated both on the premature and positive probes. The result is not yet interpretable: the count model under-fires the terminal session-end marker — positive-probe recall for `[SESSION_END: STRIKE=3: ...]` is only ≈0.06 (A8) / 0.13 (A8-trim), well below the 0.8 gate. The model largely emits the intermediate `[STRIKE=2: ...]` warning but does not escalate to the third-strike terminator. With recall this low the premature rates (0.009 vs 0.025) cannot support a trim-vs-untrim conclusion.

Two likely causes, both addressable, are deferred to follow-up work: (i) the training context window truncates long three-strike dialogues before the third-strike turn, so the model rarely sees a *labelled* session-end during training; and (ii) jointly predicting count and axis at the terminator is hard at this

scale — a sentinel+count marker without the axis label (`[SESSION_END: STRIKE=3]`) isolates the count question and should clear the recall gate. H3 therefore remains open; the core claims of this paper (H1, its mechanism, H2/H2b, and the off-family H-gen replication) do not depend on it.

5.5b Generalization — off-tutor replication (H-gen, Phase 3)

To show the trim→premature effect is a property of *next-token training on rare, count-triggered semantic markers* and not an artifact of the tutoring corpus, we replicate it in a different domain on a different model family (Appendix A). The trigger must remain semantic and recognition-gated — a literal token-counting task would let the model count exactly, leaving no recognition noise for the trim to exploit (§6.1) — so we use synthetic customer-support dialogues: the agent must emit `[ESCALATE]` after the 3rd explicit escalation request ("let me speak to a manager / a human"). The counted trigger is the *request*, so the label matches the content a reader would count; angry-but-non-requesting venting is a distractor that must not count, making per-message recognition a genuine semantic judgment rather than a keyword match. Requests are interleaved with 0–2 distractor exchanges at random positions so the trigger is decorrelated from turn index. We fine-tune **Llama-3.2-1B-Base** (a non-Qwen, full-attention base — so this run *also* speaks to the cross-family magnitude discussion of §6.6) on trimmed (dialogue ends at `[ESCALATE]`) vs untrimmed (benign resolution turns follow) data. The two corpora are generated from a single shared draw and differ *only* by the post-`[ESCALATE]` continuation (verified: the trimmed dialogue is an exact prefix of its untrimmed counterpart for all 3000 records), so the trim is the sole manipulated variable. Both are trained with identical hyperparameters and seed, with loss masked to assistant turns only (matching the tutor SFT). Premature emission is measured on held-out sub-threshold contexts (1–2 requests), with a positive-probe (3 requests) recall gate ensuring both models learned the task.

The trim effect replicates in direction (Table below). Both variants reach perfect recall on the positive probe (1.00), so the premature comparison is valid. The trimmed Llama model fires `[ESCALATE]` prematurely on sub-threshold contexts more than the untrimmed one:

task / model	premature (untrimmed)	premature (trimmed)	trim Δ	recall gate
tutor, Qwen 0.8B (A1, §5.1)	0.207	0.783	+0.576	—
support-chat, Llama-3.2-1B-Base	0.683	0.830	**+0.147**	1.00 / 1.00

Table: Off-tutor, off-family replication of the trim→premature effect. Numbers are the first-epoch checkpoint (both models pass the recall gate at 1.00); the trimmed model emits the escalation marker on sub-threshold contexts more often than the untrimmed model, the same direction as the Qwen tutor result.

The central claim is *direction*, not magnitude: the trimmed Llama model emits `[ESCALATE]` prematurely more than the untrimmed one — in a different domain, on a different family, with no tutoring or Qwen lineage in the loop — so trim→premature is a general data-shape effect, and the curation principle (§6.3) transfers to any rare, count-triggered semantic control marker. (Consistent with §5.1's primary result, the elevated *absolute* premature rate on this synthetic task — both variants are high because the sub-threshold probe is deliberately adversarial — makes the trimmed-vs-untrimmed *gap* the meaningful quantity.)

5.6 Summary

- **H1 [supported]:** trimming → premature firing, +0.44–0.58 across all four cells; dominant over position and marker.
- **H1 mechanism [supported]:** threshold-laxity on accumulated violation count, not a turn-position shortcut. Evident at the logit level (Phase 4): the trimmed model puts 34–45× more probability mass on beginning the sentinel at sub-threshold escalation than the untrimmed model (§6.1).
- **H2 [supported, single-axis]:** typed = semantic gate; 0.94–0.99 attribution, 0 contentless fires; generic cannot attribute. Attribution is trim-robust.
- **H2b [supported, Phase 1]:** attribution holds under distraction — correct-axis 0.975 (A1) / 0.963 (A5), wrong-axis <0.02, 0 contentless fires, matching the single-axis rate. The decisive semantic-gate result.
- **H-gen [supported, Phase 3]:** the trim→premature effect replicates off-tutor and off-family. On a synthetic customer-support escalation task, fine-tuning Llama-3.2-1B-Base on trimmed vs untrimmed data (identical shared draw, differing only by the post-marker continuation) yields a trimmed model that fires the escalation marker prematurely more than the untrimmed one (0.830 vs 0.683, both at recall 1.00) — the same direction as the Qwen tutor result. Trim→premature is a general data-shape effect, not a tutoring or Qwen artifact.
- **H3 [FUTURE WORK, Phase 2]:** count annotation as a candidate remedy. Design B (strike-2 warning + strike-3 session-end) was trained and evaluated, but the count model under-fires the terminal marker (positive-probe recall ≈0.06–0.13), so the trim comparison is not yet interpretable; we defer a recall-cleared version (larger context window so the third-strike turn is not truncated, and a sentinel+count marker without the axis label) to follow-up work.

6. Discussion

6.1 Why trimming causes premature firing (the mechanism)

The naive story — "trimming teaches the model the marker sits at the end of the sequence, so it fires when it detects the end" — is mechanistically incoherent: an autoregressive, strictly causal decoder generating token t conditions only on tokens 1..t-1 and has no access to whether more tokens will follow. "Am I at the end?" is not a feature it can compute at generation time. (This argument is about causality and so applies to any strictly causal decoder — linear-attention or gated-recurrent variants included — though we test the effect empirically on two families, not all architectures.)

Our evidence is most consistent with an account about which *counterexamples* trimming removes from the conditional distribution the model fits — a shortcut-learning story (Geirhos et al. 2020; McCoy et al. 2019) in which the cheapest sufficient predictor wins. We frame this as the best-supported explanation of the results below rather than a proven mechanism. SFT optimizes P(marker | left-context). The

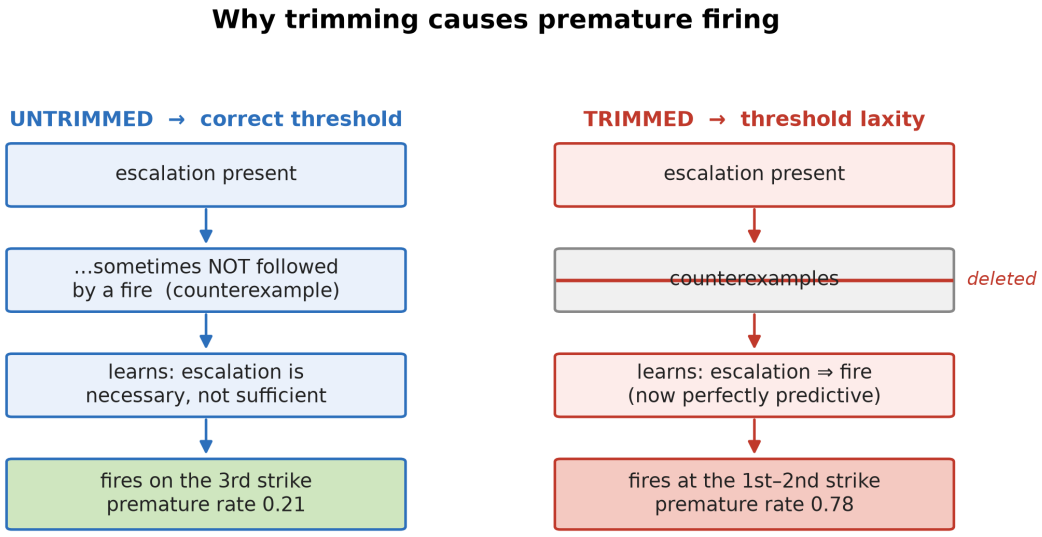

Figure 3: The mechanism. Untrimmed data contains counterexamples — escalated contexts *not* followed by a fire — so the model learns escalation is necessary but not sufficient and fires on the third strike. Trimming deletes those counterexamples, leaving escalation perfectly predictive of the marker, so the model fires early.

marker's true trigger is a *count* (third same-axis strike); the left-visible correlate the model can cheaply read is *escalation-presence* (the user is repeating a governed axis and the tutor is redirecting with escalating brevity). In untrimmed persistent records, the turns *after* the marker show an escalated context that is **not** followed by another fire — direct supervision that escalation-presence is necessary but not *sufficient*. Trimming deletes exactly those turns. After trimming, every escalated context in training terminates at a fire, so the model fits P(fire | escalation-present) ≈ 1 and fires as soon as it detects escalation — after one or two strikes — rather than counting to three.

This predicts, and the data confirm (§5.2), that premature firing rises with accumulated violation count (vc=1 → vc=2 jumps in every trimmed cell) while untrimmed cells stay low at both — the signature of escalation-thresholding failure, not a turn-position shortcut.

Semantic recognition is a precondition, not incidental. The shortcut is available only because the model's *count* of same-axis strikes is uncertain, and it is uncertain because each strike must first be *recognized* — "is this user turn a role-swap attempt?" is a fuzzy semantic judgment, not a token match. The trim does not teach "fire on a marker token seen at the end"; it teaches "fire once the *noisy accumulated evidence* of the semantic trigger is high," because trimming removed the examples showing that high evidence short of the exact threshold is not yet a fire. This is why the effect requires a semantic, recognition-gated trigger: a task that counts an *explicit, unambiguous* token would let the model count exactly and leave no recognition noise for the trim to exploit — no shortcut, no premature firing. The generalization test (§5.5b) is therefore constructed on a *different-domain but still semantic* trigger (escalation after repeated *angry* customer messages, where "angry" is recognition-gated), not on literal token counting, precisely because the recognition noise is the substrate the trim shortcut operates on.

The logit-level signature (Phase 4). The evidence supports this account directly at the logit level:

for each sub-threshold context we read the probability the model *begins the sentinel* — the joint P([) × P(SESSION | [), a two-token teacher-forced measurement (not sampling; Appendix A) that isolates the sentinel from any other bracketed token — for the A1 trim/untrim pair over the same 318 sub-threshold contexts, stratified by accumulated escalation (violation count):

violation count	untrimmed P(sentinel)	trimmed P(sentinel)	ratio
1 (one prior strike)	0.000005	0.000213	**45**×
2 (two prior strikes)	0.000045	0.001530	**34**×

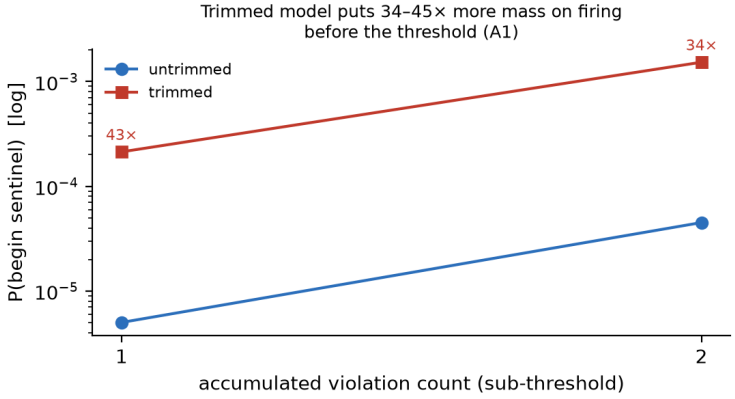

Figure 4: Logit-level signature (A1). At sub-threshold escalation the trimmed model places 34–45× more probability mass on beginning the sentinel than the untrimmed model, and that mass grows with accumulated violation count (log scale).

At sub-threshold escalation the trimmed model places **34–45× more probability mass on beginning the sentinel** than the untrimmed model, and that mass grows with accumulated escalation. The untrimmed model keeps this mass near-zero until the true third strike (consistent with its 0.207 premature rate); the trimmed model's is pushed toward the firing boundary at every sub-threshold level (consistent with its 0.783). The graded internal signal thus confirms the prose mechanism directly: trimming inflates the model's conditional probability of firing given escalation, well before the threshold. This is *not* a depth artifact. Raw turn-depth rises in lockstep with escalation (deeper turns can only carry more strikes), but the shared ~4/5 benign majority — deep dialogues that never fire — already decorrelates depth from firing; the inflated conditional therefore sits on the escalation feature specifically, which only the persistent 1/5 carries and which trimming strips of its negative half. Escalation, not depth, is the clean read.

6.2 Timing and attribution are separable

Trim collapses *timing* (premature firing) but leaves *attribution* almost untouched (A5 attribution 0.966 → 0.938 while premature 0.119 → 0.560). Marker typing governs attribution independent of timing. So "a control marker" is really two learned behaviors — a trigger (when) and a classifier (why) — with independent curation levers: the trim (or post-marker continuation) governs the trigger threshold; the marker's semantic content governs the classifier. A practitioner can get one right and the other wrong; both must be curated.

6.3 A curation principle for rare control tokens

Generalizing beyond this task: **when training a model to emit a rare marker on a threshold/count trigger, do not trim training sequences to end at the marker.** Trimming maximizes the correlation between the marker and sequence-terminality/ trigger-presence, deleting the negative examples the model needs to learn that the trigger's *presence* is not its *threshold*. Retain a benign continuation after the marker. Where the trigger is a count, additionally supervise the count explicitly in the output (§6.5). We conjecture — but do not demonstrate — that this principle extends to other rare, machine-consumed markers whose trigger is a *noisy, recognition-gated* count or threshold: refusal-after-N, escalate-after-repetition, stop-after-goal. We deliberately scope the claim this way. The mechanism (§6.1) requires that the count be *latent and uncertain* for trimming to have counterexamples worth deleting; where the trigger is exactly computable from surface tokens, there is no noise to exploit and we predict no effect (a literal token-count trigger; see the mechanism argument in §6.1). We do not run this control experiment; it follows from the mechanism rather than being established here. Whether the artifact appears for markers with crisp, non-semantic triggers — e.g. schema-driven tool/function-call or JSON control tokens emitted on an exact syntactic condition — is therefore an open empirical question we do not settle here.

6.4 Practical recommendations

The findings translate into concrete, low-cost guidance for anyone building SFT corpora that contain rare machine-consumed markers:

1. **Retain a benign continuation after the marker.** Do not trim training sequences to end at the marker; keep the post-marker turns that show an escalated context *not* followed by another fire (§5.1, §6.1). This is the single highest-leverage curation choice we identify.
2. **Treat sequence shaping as a documented experimental variable,** not a silent preprocessing detail — where a sequence is truncated can move premature firing by +0.5 (§5.1).
3. **Evaluate timing separately from semantics.** Conventional fluency/quality metrics do not detect premature firing; a dedicated sub-threshold probe does (§5.2). *When* and *why* a marker fires are separately curated behaviors (§6.2) and must be measured separately.
4. **Where the trigger is a count, supervise the count** (§6.5) rather than leaving it a latent variable the trim can corrupt.

These cost little beyond corpus bookkeeping and are independent of model scale and architecture (§6.1).

6.5 The count-annotated marker (Design B)

Trim breaks a *latent* counter; the direct remedy is to stop keeping it latent. Design B tags every strike with its running count, converting the count from an unsupervised intermediate variable into a supervised output target. To fire prematurely the model would now have to emit a wrong number (strike=2 where it should say strike=3), which the loss penalizes — so the count annotation should make firing contingent on the actual tally and, in particular, trim-robust (H3, §5.5). This is chain-of-thought/process-supervision (Nye et al. 2021; Wei et al. 2022; Lightman et al. 2024) compressed to a single tag: no separate reasoning trace, negligible inference cost, and (unlike native CoT) no delivery-at-budget failure mode. The trade-off is a changed deployment contract — the dispatcher now sees `[STRIKE: axis, N]` on non-terminal turns — which we consider acceptable because those tags are independently useful (per-turn abuse telemetry).

6.6 Threats to validity

Scope of the claims. We claim, and support with controlled experiments, a specific causal effect: for a rare marker whose trigger is a *noisy, recognition-gated* count, trimming training sequences to end at the marker inflates premature firing, via deletion of the sub-threshold counterexamples (§6.1). We do *not* claim this is a universal property of all control tokens, all curation pipelines, or all model scales. In particular the effect is demonstrated at 0.8B (Qwen3.5) plus an off-family 1B base (Llama-3.2-1B-Base, §5.5b), on count/threshold triggers, and with SFT; extrapolation to much larger models, to exactly-computable triggers (§6.3), or to RL/preference post-training is conjecture. The remaining bullets enumerate the specific axes along which our evidence is thin.

- **Single family for the main 2×2×2 — and why it is a *secondary* threat here.** All eight cells of the main design use Qwen3.5 0.8B. This matters less for paper_v3 than for a prompt-vs-train boundary claim, because our claims are about the *training signal* (which counterexamples trimming deletes; what supervising an axis label forces the model to represent), not about a specific model's capability. The generality test we run is therefore an *off-family, off-domain* replication rather than a within-corpus ablation: Phase 3 (§5.5b) fine-tunes **Llama-3.2-1B-Base** — a non-Qwen, full-attention pretrained base — on a synthetic customer-support escalation task whose trigger remains *semantic and recognition-gated* (escalate after the 3rd explicit request; angry-but-non- requesting venting is a distractor that must not count). The trim→premature effect replicates in direction there (0.683 → 0.830, both at recall 1.00), so the effect's *existence* is already family- and domain-independent. Because that run uses a full-attention model, it *also* speaks to the magnitude question: Qwen3.5 uses linear-attention/gated-recurrent layers that compress history into a fixed-size state, which could plausibly make cross-turn counting more fragile and thus *amplify* trim sensitivity relative to a full-attention model. A tighter cross-family *magnitude* comparison — re-running the exact A1 trim/untrim contrast on a small full-attention non-Qwen base (e.g. Llama-3.2-1B or SmolLM2-360M) with matched data and hyperparameters — would isolate whether the +0.5 magnitude is architecture-specific; we flag that as the open cross-family *magnitude* question while reporting *existence* as family-independent.
- **Reconstructed untrimmed cells.** The untrimmed A5/A6/A7 are rebuilt by reconstruction (Appendix A), not trained-from-scratch matched pairs. The exact-prefix verification (§4.4) is the guarantee that only the post-marker continuation differs; still, an independent from-scratch regeneration would be stronger.
- **Single dataset / task — addressed by Phase 3.** The trim principle is demonstrated primarily on the tutoring persistence task. Phase 3 (§5.5b) replicates it in a *different domain* (customer-support escalation after the 3rd angry message) on a *different model family* (Llama-3.2-1B), showing the effect is a property of next-token training on rare count-triggered semantic markers, not of this corpus or the Qwen family. The trigger is kept semantic and recognition-gated (not literal token counting) because recognition noise is the substrate the trim shortcut requires (§6.1).
- **Synthetic data — a methodological necessity, not merely a convenience.** Both our datasets are synthetic, which invites the question of whether the effect would appear on a real instruction-tuning corpus. The core of our method is a *single-variable paired contrast*: a trimmed and an untrimmed corpus that are byte-identical except for the presence of the post-marker continuation (verified as an exact prefix, §4.4). No pre-existing real dataset supplies such a pair — constructing one requires taking real dialogues and regenerating each with and without the continuation, at which point the controlled corpus is synthetic by construction. Real corpora also lack the labelled sub-threshold probes (contexts with a *known* strike count short of the threshold)

that make premature firing measurable. Synthetic control is therefore what lets us attribute the effect to the trim and nothing else; the generalization evidence we *can* give — a different domain and a different model family (§5.5b) — is the appropriate substitute for a real-corpus replication that the paired design forecloses. We none the less regard a naturalistic study (e.g. auditing an existing tool-calling corpus for trim-correlated premature calls) as valuable future work.

6.7 Future work

The most informative next experiments, in priority order:

1. **Dose-response** — vary the retained post-marker continuation (0/25/50/75/100%) to show the trim effect is continuous in counterexample supply, not a binary switch.
2. **Cross-family magnitude** — re-run the A1 contrast on a small full-attention non-Qwen base (Llama-3.2-1B / SmolLM2-360M) to test whether the +0.5 magnitude is specific to Qwen's linear-attention state; existence is already family-independent via Phase 3 (§5.5b).
3. **Scale** — repeat the contrast at 4B for scale-(in)dependence.
4. **Close H3** — the count-annotated remedy under-fires at 0.8B (§5.5); larger context and dropping the axis label are the two fixes to test.

7. Conclusion

We studied how the *shape* of SFT data governs a fine-tuned LM's emission of a rare, machine-consumed control marker, using a four-axis session-ending sentinel as a controlled testbed.

Our primary result is counter-intuitive and robust: **trimming training sequences to end at the marker — a curation step one might expect to sharpen marker learning — instead induces premature firing.** Across every (position × marker) cell, trimming raised the premature-firing rate by +0.44 to +0.58, a main effect far larger than the position or marker design choices it is usually bundled with. The mechanism is not a turn-position shortcut and not an incoherent "detects the end" story; it is threshold-laxity: trimming deletes the training examples in which an escalated context is *not* followed by a fire, leaving escalation-presence perfectly predictive of the marker and impairing the model's ability to threshold on the strike count. Retaining the benign post-marker continuation restores correctly-timed firing.

We further showed that *when* a model fires and *why it says it fires* are separable, separately-curated behaviors: **typed markers act as semantic gates**, yielding 0.94–0.99 correct-axis attribution and zero contentless fires, where generic markers cannot attribute at all — and attribution is robust to the trim that collapses timing. The gate holds under a distractor axis (correct-axis 0.96–0.98, pulled to the distractor <2%), so the label forces a genuine per-axis semantic check. Finally, the trim effect is **not corpus-specific**: it replicates off-family and off-domain — fine-tuning Llama-3.2-1B-Base on a synthetic customer-support escalation task reproduces trim→premature (trimmed 0.830 vs untrimmed 0.683, both at full recall) — establishing it as a general property of next-token training on rare, count-triggered semantic markers. As a candidate remedy we sketch **count-annotated markers** — supervising the running strike count in the output, predicted to make firing trim-robust — and leave a recall-cleared evaluation of them to follow-up work.

The practical takeaway is a curation principle for any rare control token with a count/threshold trigger: **do not trim to the marker; retain a continuation after it; and where the trigger is a count, supervise the count explicitly.** These are cheap, low-overhead data choices with first-order effects on

control-token reliability, and the effect is observed across two model families (Qwen3.5, Llama-3.2).

Appendix A. Reproducibility

Every result in the paper is produced by a named script over generations already on disk (no result depends on unseen inference). Base model: Qwen3.5-0.8B-Base, QLoRA SFT, 1-epoch matched budget unless noted; the off-family replication (§5.5b) uses Llama-3.2-1B-Base. The table below maps each reported result to the script that computes it and the JSON it writes.

Result (section)	Script	Output
Trim × (position×marker) premature rates (§5.1)	`scripts/score_phase0_attribution.py`	`outputs/paper_v2/score/phase0_a`
A1 positive-probe recall gate, 1-epoch (§5.1)	`scripts/run_paper_eval.py --baseline v3_a1_{untrim,trim} --test-set persistent_probe`	`outputs/paper_v3/eval_recall_ch`
Significance: Fisher / McNemar / Wilson CIs + effect sizes (odds ratio, Cohen's *h*) (§5.1)	`scripts/score_phase0_significance.py` (stdlib only)	`outputs/paper_v2/score/phase0_s`
Seed-7 replication (§5.1)	`scripts/run_multiseed_s7.py` → `scripts/score_multiseed_s7.py`	`outputs/paper_v2/score/phase0_m`
vc-stratified premature (§5.2)	`scripts/score_phase0_attribution.py`	`outputs/paper_v2/score/phase0_a`
Logit-level P(sentinel) vs depth (§6.1)	`scripts/score_sentinel_logits_depth.py`	`outputs/paper_v3/score/sentinel`
Single-axis typed attribution / semantic gate (§5.3)	`scripts/score_phase0_attribution.py`	`outputs/paper_v2/score/phase0_a`
Attribution under distraction / mixed-violation probe (§5.4)	`scripts/score_mixed_violation_probe.py`	`outputs/paper_v3/score/mixed_vi`
Count-marker (Design B) construction (§5.5)	`scripts/convert_typed_to_count.py`, `scripts/assemble_a8_count.py`	`data/sft_filtered_a8_count*`
Off-family Llama replication (§5.5b)	`scripts/phase3_synthetic_pipeline.py`	`outputs/paper_v3/phase3/phase3_`
Untrimmed A5/A6/A7 reconstruction (§4.4, §6.6)	`scripts/build_untrim_a5a6a7.py`	`data/paper_v2/phase0_trim_st`

The firing detector used throughout is the axis-capturing regex `\[SESSION_END(?:\s*:\s*(?:STRIKE=\d+\s*:\s*)?` which matches generic, typed, and count-annotated marker forms and captures the axis label for attribution. Code, configs, and the synthetic datasets are in the released repository (see Code and Data Availability); model weights and large training outputs are excluded.

Code and Data Availability. All code, per-condition training configurations, seeds, frozen evaluation sets, judge prompts, the synthetic datasets, and the score outputs underlying every table are released at https://github.com/cch-ai922/tutor-train. A `reproducibility/` guide maps each result to the script, config, and expected number that produce it. The off-family replication (§5.5b) uses Llama-3.2-1B-Base; trained LoRA adapters are low-rank deltas over the public base models and are available from the authors on request.

Appendix B. Supporting tables

Table B.1 — Premature firing stratified by violation count (§5.2). vc = number of prior same-axis sub-threshold strikes. In every trimmed cell firing jumps sharply from vc=1 to vc=2; untrimmed cells stay low at both — the threshold-laxity signature.

Cell	variant	vc=1	vc=2
A1	untrimmed	0.063	0.352
A1	trimmed	0.654	0.912
A5	untrimmed	0.031	0.207
A5	trimmed	0.384	0.736
A6	untrimmed	0.050	0.264
A6	trimmed	0.497	0.786
A7	untrimmed	0.069	0.358
A7	trimmed	0.497	0.887

Table B.2 — Attribution under distraction (§5.4). `fire_correct` records (primary axis X at the third strike, distractor axis Y sub-threshold), typed cells only (n=141 each). The gate names the threshold axis X on 0.96–0.98 of fires and is pulled to the distractor under 2%.

Cell	marker	fire_rate (X at 3)	correct-axis (X)	wrong-axis (Y)	no-axis fires
A1	typed	0.837	**0.975**	0.009	0
A5	typed	0.766	**0.963**	0.018	0

Code and Data Availability

Code, training/evaluation scripts, configuration, and the synthetic datasets are available at https://github.com/cch-ai922/tutor-train. Model weights and large training outputs are not included; the base model is Qwen3.5-0.8B-Base. The released datasets are model-generated (teacher-distilled) and contain no personal data.

References

Bhattamishra, Satwik, Kabir Ahuja, and Navin Goyal. 2020. "On the Ability and Limitations of Transformers to Recognize Formal Languages." *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 7096–116.

Dettmers, Tim, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. 2023. "QLoRA: Efficient Finetuning of Quantized LLMs." *Advances in Neural Information Processing Systems (NeurIPS)*.

Dong, Yixin, Charlie F. Ruan, Yaxing Cai, et al. 2024. "XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models." *arXiv Preprint arXiv:2411.15100*.

Geirhos, Robert, Jörn-Henrik Jacobsen, Claudio Michaelis, et al. 2020. "Shortcut Learning in Deep Neural Networks." *Nature Machine Intelligence* 2 (11): 665–73.

Gururangan, Suchin, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. 2018. "Annotation Artifacts in Natural Language Inference Data." *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 107–12.

Hu, Edward J., Yelong Shen, Phillip Wallis, et al. 2022. "LoRA: Low-Rank Adaptation of Large Language Models." *International Conference on Learning Representations (ICLR)*.

Lightman, Hunter, Vineet Kosaraju, Yura Burda, et al. 2024. "Let's Verify Step by Step." *International Conference on Learning Representations (ICLR)*.

Liu, Weiwen, Xu Huang, Xingshan Zeng, et al. 2025. "ToolACE: Winning the Points of LLM Function Calling." *International Conference on Learning Representations (ICLR)*.

Liu, Zuxin, Thai Hoang, Jianguo Zhang, et al. 2024. "APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets." *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

McCoy, Tom, Ellie Pavlick, and Tal Linzen. 2019. "Right for the Wrong Reasons: Diagnosing Syntactic Heuristics in Natural Language Inference." *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 3428–48.

Muennighoff, Niklas, Alexander M. Rush, Boaz Barak, et al. 2023. "Scaling Data-Constrained Language Models." *Advances in Neural Information Processing Systems (NeurIPS)*.

Nye, Maxwell, Anders Johan Andreassen, Guy Gur-Ari, et al. 2021. "Show Your Work: Scratchpads for Intermediate Computation with Language Models." *arXiv Preprint arXiv:2112.00114*.

Park, Kanghee, Jiayu Wang, Taylor Berg-Kirkpatrick, Nadia Polikarpova, and Loris D'Antoni. 2024. "Grammar-Aligned Decoding." *Advances in Neural Information Processing Systems (NeurIPS)*.

Qin, Yujia, Shihao Liang, Yining Ye, et al. 2024. "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-World APIs." *International Conference on Learning Representations (ICLR)*.

Rafailov, Rafael, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon, and Chelsea Finn. 2023. "Direct Preference Optimization: Your Language Model Is Secretly a Reward Model." *Advances in Neural Information Processing Systems (NeurIPS)*.

Schick, Timo, Jane Dwivedi-Yu, Roberto Dessì, et al. 2023. "Toolformer: Language Models Can Teach Themselves to Use Tools." *Advances in Neural Information Processing Systems (NeurIPS)*.

Wei, Jason, Xuezhi Wang, Dale Schuurmans, et al. 2022. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." *Advances in Neural Information Processing Systems (NeurIPS)*.

Willard, Brandon T., and Rémi Louf. 2023. "Efficient Guided Generation for Large Language Models." *arXiv Preprint arXiv:2307.09702*.

Yao, Shunyu, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. "τ-bench: A Benchmark for

Tool-Agent-User Interaction in Real-World Domains." *arXiv Preprint arXiv:2406.12045*.

Zheng, Chujie, Zhenru Zhang, Beichen Zhang, et al. 2024. "ProcessBench: Identifying Process Errors in Mathematical Reasoning." *arXiv Preprint arXiv:2412.06559*.

Zhou, Chunting, Pengfei Liu, Puxin Xu, et al. 2023. "LIMA: Less Is More for Alignment." *Advances in Neural Information Processing Systems (NeurIPS)*.